FINAL PROJECT · ADVANCED TECHNOLOGIES

# Smart Parking Lot

Intelligent vehicle access control system with clean architecture, SOLID principles and IoT hardware support

- C# 14 / .NET 10
- Clean Architecture
- Arduino
- SQLite


Santiago Barrera Meza 160005004
Jesus Gregorio Delgado 160005010
Johan Sebastian Forero Agudelo 160005011
David Andres Monroy Aguiar 160005019

CONTEXT

# Problem Statement

Traditional parking lots rely on **manual processes and human supervision** to record vehicle entry and exit, determine space availability, and control access.

This creates **operational bottlenecks**, registration errors, long wait times and a poor user experience, along with unnecessary staffing costs.

The absence of an automated system makes **real-time monitoring**, vehicle traceability and data-driven decision making impossible.

### Inefficient Manual Registration

Without automation, vehicle entry and exit depends on operators and error-prone physical forms.

### No Real-Time Visibility

Drivers don't know how many spaces are available before entering, causing congestion and wasted time.

### Lack of Traceability

No plate-based access history exists, making audits, accurate billing and usage reporting impossible.

REQUIREMENTS

# Use Cases Table

| Use Case | Sensor(s) / Peripheral(s) | Trigger Condition | Expected System Reaction | Data to Persist | Done? |
|---|---|---|---|---|---|
| **Parking Occupancy Detection** | IR Sensor, ArduinoSerialBridge, LED | Arduino sends a SpotSensorReading via IArduinoReader | Captures reading, updates spot state & parking availability | Spot ID, occupancy status, timestamp | ✓ Yes |
| **Entry Control (LPR)** | Camera Module, ArduinoSerialBridge, Servo Motor | Arduino sends GateSensorReading with detected plate | Creates EntryRequest, checks capacity, opens gate if approved | Plate, entry timestamp, approval status | ✗ No |
| **Exit Control (LPR)** | Camera Module, ArduinoSerialBridge, Servo Motor | Arduino sends GateSensorReading at exit gate | Creates ExitRequest, releases spot, opens gate | Plate, exit timestamp, released spot ID | ✗ No |
| **Full Capacity Alert** | IR Sensors, ArduinoSerialBridge, LED | CapacityService finds no spots after processing SpotSensorReading | Generates alert via AlertService, blocks new EntryRequests | Alert ID, type, message, timestamp | ✗ No |

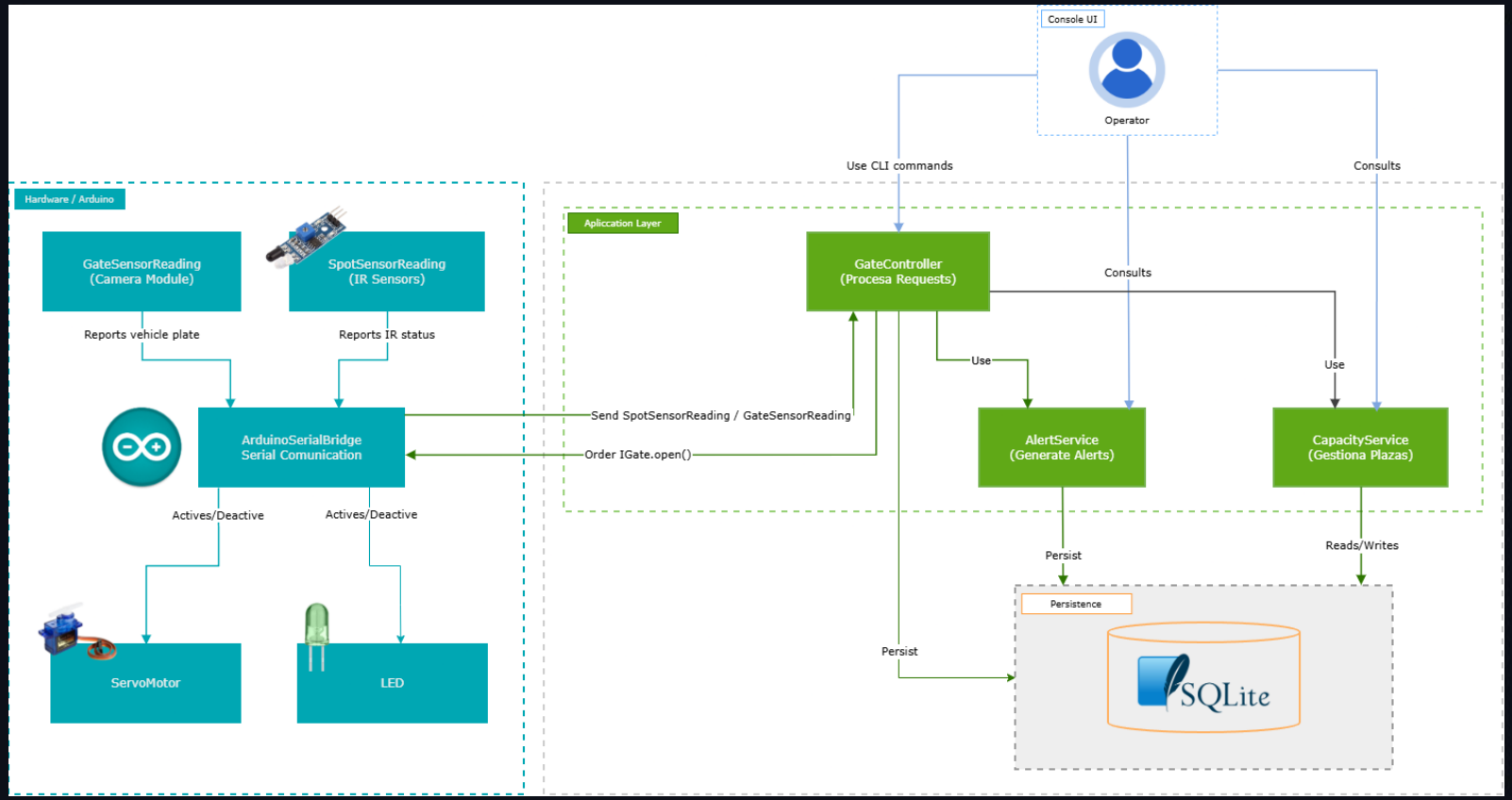
HARDWARE

# Materials Matrix

| Component | Qty | Role in the Project |
|---|---|---|
| **Arduino Uno** | 2 | Processes sensor logic and dispatches commands to actuators |
| **IR Sensor (Infrared)** | 8 | Detects vehicle presence in each parking spot |
| **Servo SG90** | 2 | Opens and closes the physical access barriers (entry & exit gates) |
| **Camera OV7670** | 2 | Captures vehicle plates for License Plate Recognition (LPR) at entry & exit |
| **LEDs (Red / Green)** | 2 | Visual guidance for driver — indicates spot availability |
| **Jumper Wires** | 30+ | Hardware interconnection between all components |

DESIGN

# System Architecture

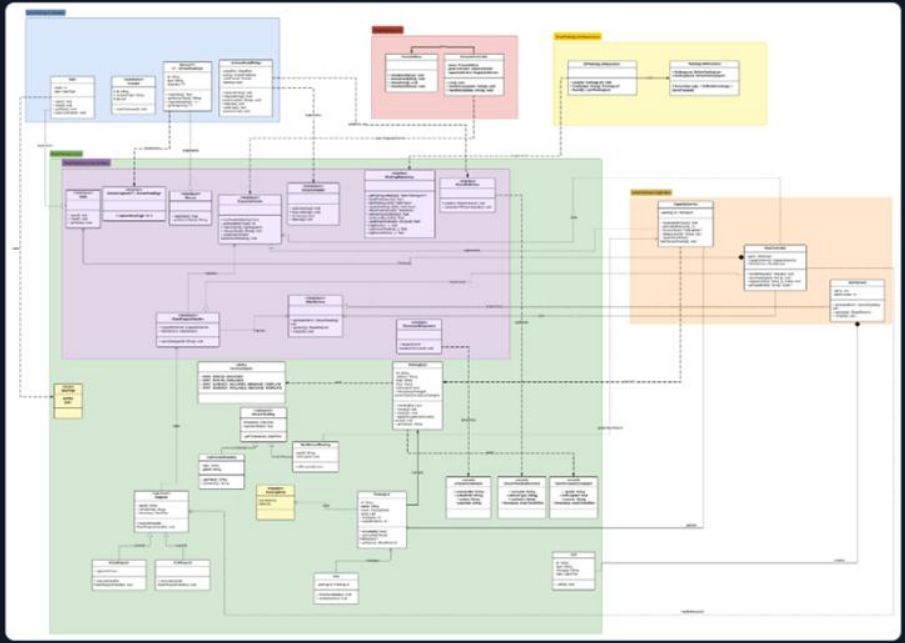


MODELING

# UML Class Diagram

Represents the static structure of the system — classes, interfaces and their relationships across the four-layer clean architecture.

- **Core**
  Entities & interfaces — zero dependencies
- **Application**
  Controllers and application services
- **Hardware**
  Sensors, actuators and physical gate
- **CLI**
  Composition root & dependency injection



SOFTWARE DESIGN

# SOLID Principles Applied

## S

### Single Responsibility

Each class has one clearly defined responsibility within the system.

```
ParkingSpot → state
GateController → flow
CapacityService → capacity
```

## O

### Open / Closed

Abstractions allow extending the system without modifying existing code.

```
IGateRequestHandler
allows new
implementations
```

## L

### Liskov Substitution

Concrete implementations are interchangeable with their interfaces without breaking the system.

```
Gate : IGate
Sensor : ISensor
are substitutable
```

## I

### Interface Segregation

Small, focused interfaces — no class is forced to implement methods it doesn't need.

```
ICapacityService
IAlertService
IGate · ISensor
```

## D

### Dependency Inversion

High-level modules depend on abstractions defined in Core, not on concrete implementations.

```
Core → 0 deps
Application →
Core
Hardware → Core
```

INTERFACE

# Application Menu

The application exposes an interactive console menu (CLI) to operate the parking lot, monitor sensors and query history from the terminal.

Options marked with # are features under development not yet implemented in the current demo.

SmartParkingLot.Cli

```
╔═══════════════════════════════════════╗
‖ Smart Parking Lot — Menú ‖
╚═══════════════════════════════════════╝

Parking Lot : Main Parking Lot (LOT-001)
Mode : Standard
Spots : 3 total | 2 available
# → Not implemented in this demo

1. # Request vehicle entry
2. # Request vehicle exit
3. # Update spot status (manual sensor)
4. View parking lot status
5. # View vehicle history
6. View sensor readings
7. # View device actions
8. Live monitoring (Arduino)
9. View spot status
0. Exit
```

ROADMAP

# Future Implementations

# 1

### Vehicle Entry Request

Integration with License Plate Recognition (LPR) to automatically authorize entry, log the event and assign an available spot in real time.

# 2

### Vehicle Exit Request

Automatic exit detection via gate sensor, spot release, stay duration calculation and payment receipt generation.

# 3

### Spot Update via Sensor

Automatic readings from ultrasonic or IR sensors to update each spot's occupancy state without human intervention.

# 5

### Vehicle History

Full access history by plate: dates, entry/exit times, assigned spots and approval status for each request.

# 7

### Device Actions Log

Recording and display of the action log for each physical actuator (gates, LEDs), useful for hardware audits and fault diagnosis.

+

### Desktop GUI

A visual Desktop layer added to the architecture, preserving the Dependency Inversion Principle by depending only on Core abstractions.

STACK

# Technologies Used

C#

## C# 14

Lenguaje principal

Modern syntax with Records, Pattern Matching, top-level statements and nullable reference types for compile-time safety.

Records · Pattern Matching · Nullable

.NET

## .NET 10

Runtime & SDK

Cross-platform runtime. Solution with four independent projects interconnected via project references.

Multi-proyecto · DI Manual

## Arduino

Hardware IoT

Serial communication with microcontroller for IR sensor readings and LED actuator control from C# via COM port.

Serial Bridge · Sensores IR · LEDs

## SQLite

Persistencia

Serverless embedded database to persist parking lot state, request history, sensor readings and device actions.

Embebida · Sin servidor

# Thanks!

Smart Parking Lot — Advanced Technologies

- C# 14 / .NET 10
- Clean Architecture
- SOLID & GRASP
- Arduino + SQLite